MACHINE LEARNING • MODULE 10

ML Project Workflow

End-to-End Machine Learning in Production

■ ML PROJECTS

■ LEARNING OUTCOME

By the end of this module you will structure an end-to-end ML project, track experiments with MLflow, deploy a model as a REST API with FastAPI, monitor for drift in production, and create a portfolio-worthy GitHub project.

■ Skills You Will Gain

- Structure a complete ML project with clear phases
- Track all experiments with MLflow: params, metrics, artifacts
- Deploy a trained model as a FastAPI REST endpoint
- Save and load models with pickle and joblib
- Monitor for data drift and model degradation in production
- Document and present projects for GitHub portfolio

■ **A deployed ML model gets 3x more interview callbacks than a notebook. The ability to ship models — not just train them — is what separates senior ML engineers from juniors.**

SECTION 1

Complete ML Project Checklist

1	**Problem Framing** Business goal -> ML task -> success metric -> minimum viable performance threshold
2	**Data Acquisition** Collect, validate, version data. Document source, license, and collection date.
3	**EDA** Minimum 10 visualisations. Find patterns, outliers, class balance, correlations.
4	**Feature Engineering** Create, encode, scale. Justify every decision with evidence from EDA.
5	**Baseline Model** Simple model first (LogReg, mean). Beat this before adding complexity.
6	**Model Experiments** Try 3-5 algorithms. Track ALL experiments with MLflow.
7	**Hyperparameter Tuning** Optuna or GridSearch. Proper cross-validation. Never touch test set.
8	**Final Evaluation** Confusion matrix, ROC, SHAP explanations. Test set used exactly ONCE.
9	**Deployment** FastAPI endpoint. Docker container. CI/CD pipeline.
10	**Monitoring** Log predictions. Alert on data drift. Schedule periodic retraining.

SECTION 2

MLflow Experiment Tracking

```
import mlflow import mlflow.sklearn mlflow.set_experiment('customer_churn_prediction') with
mlflow.start_run(run_name='xgboost_v3_tuned'): # Log hyperparameters
mlflow.log_params({'n_estimators': 300, 'learning_rate': 0.05, 'max_depth': 5, 'subsample': 0.8}) #
Train model model.fit(X_train, y_train) y_pred = model.predict(X_test) y_proba =
model.predict_proba(X_test)[:, 1] # Log metrics mlflow.log_metrics({'f1': f1_score(y_test,
y_pred),'auc': roc_auc_score(y_test, y_proba), 'precision': precision_score(y_test,
y_pred),'recall': recall_score(y_test, y_pred)}) # Log model and artifacts
mlflow.sklearn.log_model(model, 'xgb_model') mlflow.log_artifact('confusion_matrix.png')
mlflow.log_artifact('shap_summary.png') # Launch MLflow UI: mlflow ui -> open http://localhost:5000
```

SECTION 3

FastAPI Model Deployment

```
# app.py from fastapi import FastAPI from pydantic import BaseModel import pickle, pandas as pd app
= FastAPI(title='Churn Prediction API', version='1.0') model = pickle.load(open('model.pkl', 'rb'))
```

```
scaler = pickle.load(open('scaler.pkl', 'rb')) class CustomerFeatures(BaseModel): tenure: float
monthly_charges: float contract_type: str # 'Month-to-month', 'One year', 'Two year' num_services:
int @app.post('/predict') def predict_churn(data: CustomerFeatures): df =
pd.DataFrame([data.dict()]) # encode contract_type -> numeric, etc. df_scaled =
scaler.transform(df[feature_cols]) prob = model.predict_proba(df_scaled)[0][1] return {
'churn_probability': round(float(prob), 4), 'risk_level': 'HIGH' if prob > 0.5 else 'LOW',
'recommendation': 'Trigger retention offer' if prob > 0.5 else 'No action needed' }
@app.get('/health') def health(): return {'status': 'healthy'} # Run: uvicorn app:app --reload #
Test: http://localhost:8000/docs (auto Swagger UI)
```

SECTION 4

GitHub Portfolio Structure

```
# Recommended project folder structure customer-churn-prediction/ README.md # problem + results +
demo link + setup steps requirements.txt # pip install -r requirements.txt data/ raw/ # original
unmodified data processed/ # cleaned and feature-engineered notebooks/ 01_EDA.ipynb
02_Feature_Engineering.ipynb 03_Modelling.ipynb src/ preprocess.py # reusable preprocessing
functions train.py # training script (run to reproduce results) predict.py # inference utilities
api/ app.py # FastAPI app Dockerfile # containerise for deployment models/ model.pkl # saved
trained model scaler.pkl # saved fitted scaler tests/ test_predict.py # unit tests
```

SECTION 5

ML Interview Q&A;

Question	Strong Answer
Walk me through an end-to-end ML project	Problem -> Data -> EDA -> FE -> Baseline -> Experiments -> Tuning -> Eval -> Deploy.
Why did you choose XGBoost over Random Forest?	XGBoost is boosting (sequential correction) vs RF bagging (parallel). On this tabular dat
How did you handle class imbalance?	Detected in EDA: 85/15 split. Used class_weight=balanced + SMOTE + evaluated with F
Explain a model's prediction to a business stakeholder	SHAP shows this customer churned because monthly charge was high AND on month-t
What would you improve with more time?	Collect more signals (browsing data, support tickets), try deep tabular models (TabNet),

■ PRACTICE Q & A

Q1. Why should you always build a baseline model first?

A: *Sets a performance floor — if your complex model doesn't beat a simple mean predictor, something is wrong with your data or approach. Also helps justify the complexity.*

Q2. What does MLflow track and why is it essential?

A: *Parameters, metrics, artifacts (plots, models), code version, and run time. Lets you compare all experiments reproducibly and roll back to any previous version.*

Q3. What is model drift and how do you detect it?

A: *When real-world data distribution shifts over time, model performance degrades. Detect by monitoring prediction distribution, input feature statistics, and periodic accuracy checks on labelled samples.*

Q4. Why use FastAPI over Flask for ML model serving?

A: *FastAPI is async, auto-validates inputs with Pydantic, generates OpenAPI (Swagger) docs automatically, and is 2-3x faster than Flask. Pydantic models catch bad inputs before they reach the model.*

Q5. What must a good ML project README include?

A: *Problem statement, dataset description, approach and key decisions, results with numbers, how to reproduce, limitations, and links to notebook/demo/deployed API.*

■ ML Workflow Cheat Sheet

`mlflow.start_run()`	Begin experiment tracking
`mlflow.log_params({})`	Record hyperparameters
`mlflow.log_metrics({})`	Record performance metrics
`mlflow.sklearn.log_model(m)`	Save model to MLflow
`pickle.dump(model, f)`	Save model to disk
`pickle.load(open('m.pkl','rb'))`	Load model from disk
`FastAPI() + @app.post`	Create prediction endpoint
`uvicorn app:app --reload`	Run FastAPI dev server
`BaseModel (Pydantic)`	Input schema + validation
`docker build -t ml-api .`	Build container image
`SHAP TreeExplainer`	Explain tree model predictions
`Evidently AI`	Production model monitoring